# Calendar ChatBot – Project Report

Oct 11, 2026 · Iqra Aiman · Roll No. 160125771093

## 1. Introduction

Calendar ChatBot is a Java desktop application that lets a user manage personal reminders by chatting. The user types plain commands such as `schedule` or `show schedule`, and the bot replies in a chat window. When a saved task's time arrives, the application beeps and shows a pop-up alert.

The project was built for the Java Programming course as one of the listed applications (Basic Chatbot), extended with scheduling. It uses only the free Java Development Kit: Swing for the interface, `java.time` for dates, `java.util` collections for storage in memory, and `java.util.concurrent` for the background alert checker. Tasks are saved to a text file, so they remain after the program is closed.

## 2. Objectives

1. Build a Swing chat window with a scrollable message area, a text input field and a Send button.
2. Let the user schedule a task through a short conversation: task name, then date, then time.
3. List saved tasks with `show schedule` and remove them with `delete [task]`.
4. Alert the user with a sound and a pop-up when a reminder is due, using `ScheduledExecutorService`.
5. Reject invalid or past dates and times without crashing, using exception handling.
6. Store tasks in an `ArrayList` and save them to a text file so they persist.
7. Organise the code into separate classes with clear responsibilities, applying OOP principles.

## 3. System design

The system has five classes in three layers: presentation (`ChatWindow`), logic (`ChatBot`) and data (`Reminder`, `ReminderStorage`), started by `Main`.

## 3.1 Class diagram description

| Class | Role | Key attributes | Key methods |
|---|---|---|---|
| `Main` | Entry point; creates and connects the objects | none | `main(String[] args)` |
| `ChatWindow` (extends `JFrame`) | GUI: chat bubbles, input, alerts | `bot: ChatBot`, `messagesPanel: JPanel`, `inputField: JTextField`, `sendButton: JButton`, `scheduler: ScheduledExecutorService` | `sendMessage()`, `startReminderChecker()`, `checkReminders()`, `addMessage(String, boolean)`, `closeApp()` |
| `ChatWindow.Bubble` (extends `JLabel`, inner class) | Draws one rounded message bubble | `fill: Color` | `paintComponent(Graphics)`, `toHtml(String)` |
| `ChatBot` | Logic: commands, scheduling conversation, task list | `reminders: ArrayList<Reminder>`, `storage: ReminderStorage`, `state: State`, `pendingName: String`, `pendingDate: LocalDate` | `handleInput(String)`, `receiveName()`, `receiveDate()`, `receiveTime()`, `showSchedule()`, `deleteTask(String)`, `collectDueReminders(LocalDateTime)` |
| `ChatBot.State` (enum) | Tracks the step of the scheduling conversation | `IDLE`, `ASK_NAME`, `ASK_DATE`, `ASK_TIME` | none |
| `Reminder` | Model: one task | `taskName: String`, `dueTime: LocalDateTime`, `alerted: boolean` | getters, `setAlerted(boolean)`, `isDue(LocalDateTime)`, `toString()` |
| `ReminderStorage` | File handling for `tasks.txt` | `file: File` | `load(): List<Reminder>`, `save(List<Reminder>)` |

## 3.2 Relationships

- **Inheritance:** `ChatWindow` is a `JFrame`; `Bubble` is a `JLabel`.
- **Association:** `ChatWindow` holds one `ChatBot`; `ChatBot` holds one `ReminderStorage`.
- **Aggregation:** `ChatBot` contains zero or more `Reminder` objects in its `ArrayList`.
- **Dependency:** `Main` creates `ReminderStorage`, `ChatBot` and `ChatWindow`; `ReminderStorage` creates `Reminder` objects when loading.

## 3.3 Data flow

User input goes from `ChatWindow` to `ChatBot.handleInput()`, which returns a reply string for display. Every change to the task list is written to `tasks.txt` through `ReminderStorage.save()`. A background thread asks `ChatBot` every 10 seconds for due reminders, and `ChatWindow` displays the alerts.

# 4. Implementation details

**Tools:** Java 21 (OpenJDK; the code also compiles on Java 8 and later), Swing, command-line `javac` and `java`. No third-party libraries.

**Chat interface.** `ChatWindow` uses a `BorderLayout`: a ruby header (#9B111E) at the top, a scrollable message area in the centre, and the input row at the bottom. Messages are rounded bubbles drawn by overriding `paintComponent()`; user bubbles sit on the right in a light ruby tint, bot bubbles on the left in white. The background is lavender (#E6E6FA) and text is dark grey (#2B2B2B) for readability. Long messages wrap using HTML inside a `JLabel`.

**Event handling.** The Send button and the Enter key share one handler through `ActionListener` lambdas. A `WindowAdapter` makes sure the background thread stops when the window is closed.

**Conversation state.** Scheduling needs three answers, so `ChatBot` keeps a `State` enum. A `switch` in `handleInput()` sends each message to the right step. Typing `cancel` returns to `IDLE` at any step.

**Date and time validation.** Dates use the pattern `dd-MM-uuuu` and times `HH:mm`, both with `ResolverStyle.STRICT`, so impossible values such as 31-02-2026 or 25:00 throw `DateTimeParseException`. The exception is caught and the bot asks again. Dates or times in the past are also rejected.

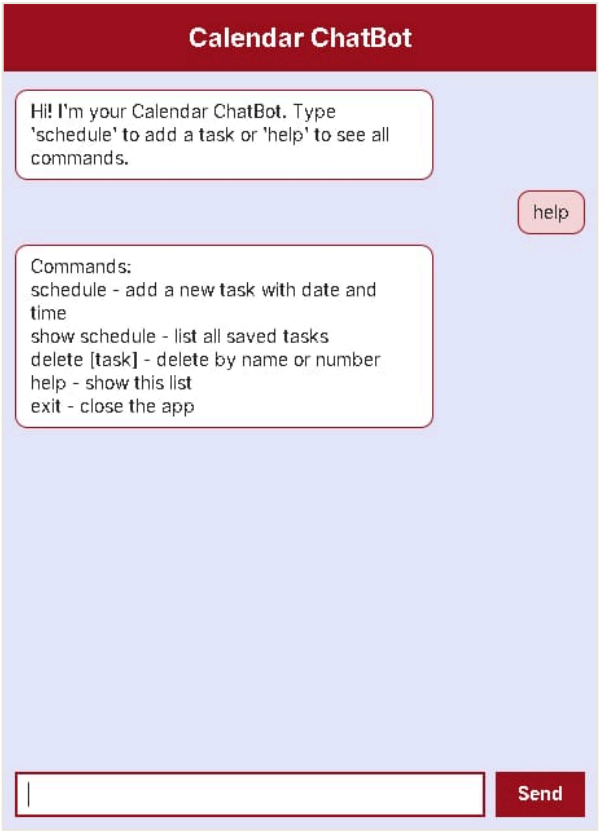
**Reminder alerts.** A single-thread `ScheduledExecutorService` runs every 10 seconds. It uses `SwingUtilities.invokeLater` to run `checkReminders()` on the Event Dispatch Thread, because Swing is not thread-safe. Each due reminder triggers `Toolkit.beep()`, a chat message, and a `JOptionPane` dialog, then is marked `alerted` so it never fires twice.

**Persistence.** `ReminderStorage` writes one line per task (`name|date-time|alerted`) using `BufferedWriter`, and reads the file back with `BufferedReader` on start-up. Both use try-with-resources, and broken lines are skipped instead of crashing the program. The file is rewritten after every change.

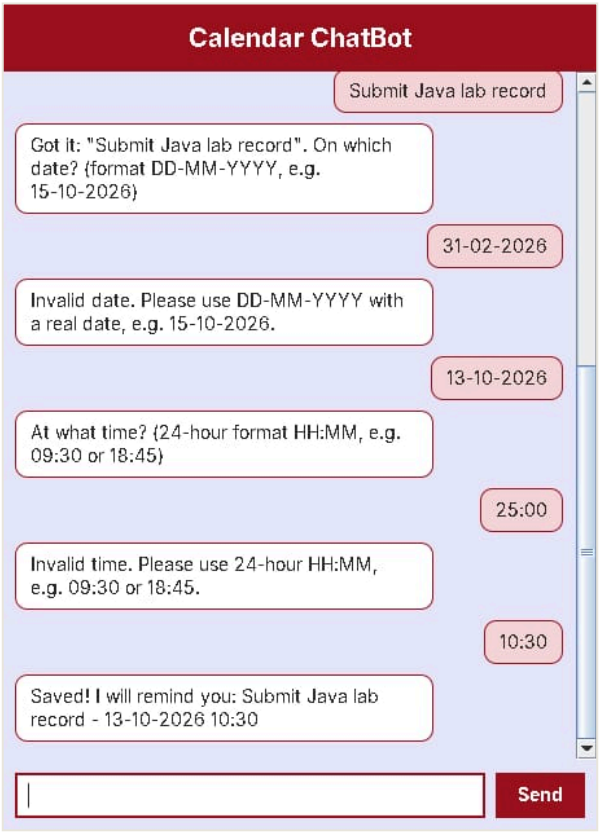
**Delete.** `delete` accepts a task number from `show schedule` or a task name (case-insensitive). The list is kept sorted by due time, so the numbers always match the displayed order.

# 5. Screenshots

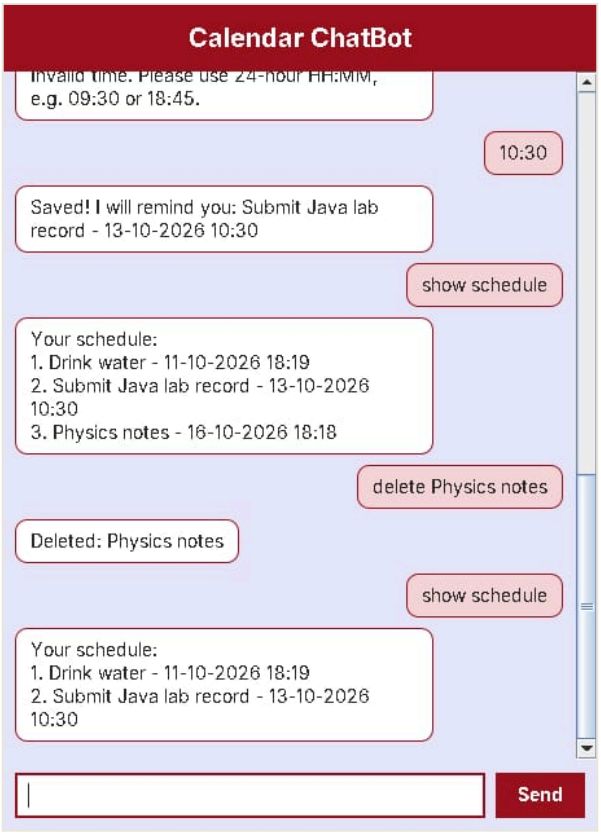
## Figure 1: Start screen and help command




## Figure 2: Scheduling a task with validation



## Figure 3: Show schedule and delete




## Figure 4: Reminder alert

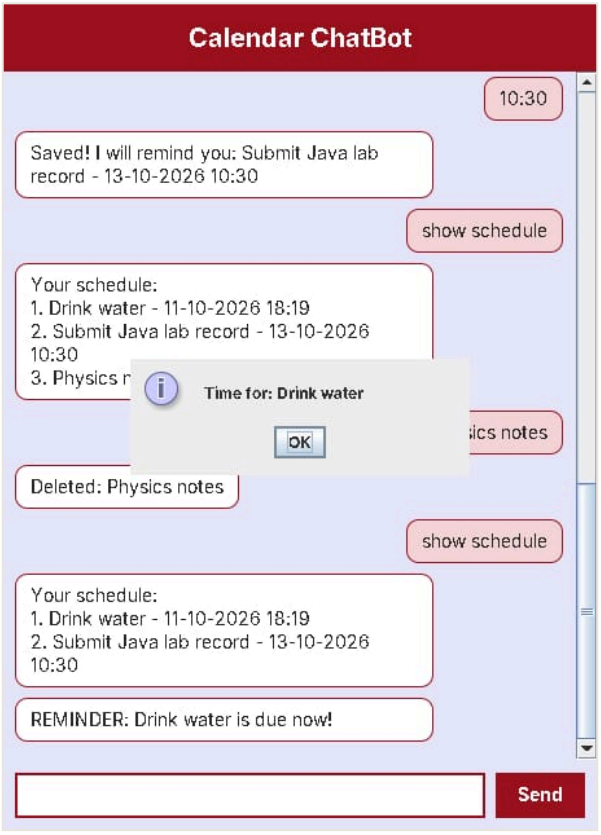
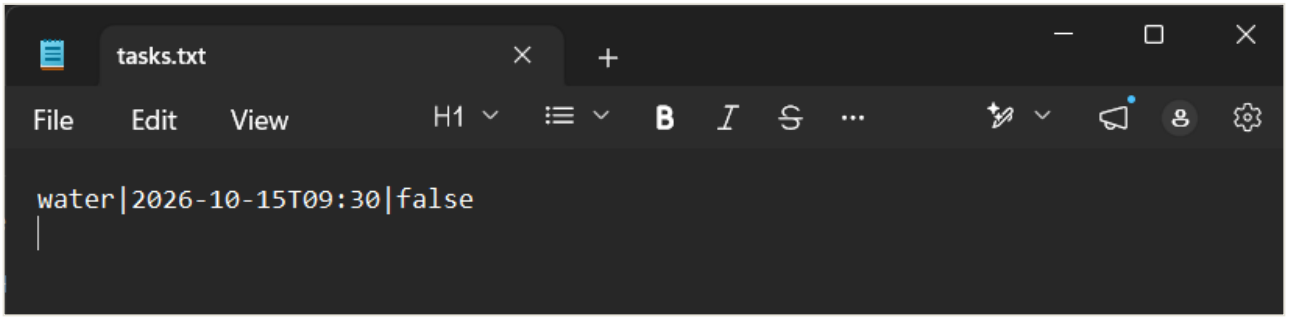


**Figure 5: Saved tasks file**




## 6. Testing results

All 24 test cases passed. The logic was tested with an automated test class that sends messages to `ChatBot` and checks each reply. The GUI was tested by running the full application and typing the same commands, including waiting for a live reminder pop-up.

| # | Test case | Input | Expected result | Result |
|---|---|---|---|---|
| 1 | Help command | `help` | List of all commands | Pass |
| 2 | Empty schedule | `show schedule` with no tasks | "Your schedule is empty" | Pass |
| 3 | Unknown command | `blah` | "Sorry, I don't know that command" | Pass |
| 4 | Start scheduling | `schedule` | Bot asks for the task name | Pass |
| 5 | Task name | `Maths exam` | Bot asks for the date | Pass |
| 6 | Date as words | `tomorrow` | "Invalid date", asks again | Pass |
| 7 | Impossible date | `31-02-2027` | "Invalid date", asks again | Pass |
| 8 | Invalid month | `10-13-2027` | "Invalid date", asks again | Pass |
| 9 | Past date | `01-01-2020` | "That date is in the past" | Pass |

| # | Test case | Input | Expected result | Result |
|---|---|---|---|---|
| 10 | Valid date | a date 3 days ahead | Bot asks for the time | Pass |
| 11 | Invalid hour | `25:00` | "Invalid time", asks again | Pass |
| 12 | Wrong time format | `9.30` | "Invalid time", asks again | Pass |
| 13 | Valid time | `09:30` | "Saved!" with task details | Pass |
| 14 | Time already passed today | today's date, then `00:00` | "That time has already passed" | Pass |
| 15 | Cancel | `cancel` during scheduling | "Scheduling cancelled" | Pass |
| 16 | Second task | a task for tomorrow, `18:45` | "Saved!" | Pass |
| 17 | Sorted list | `show schedule` | Earliest task listed first | Pass |
| 18 | Persistence | restart the bot, `show schedule` | Both tasks loaded from `tasks.txt` | Pass |
| 19 | Delete by name | `delete maths EXAM` | "Deleted: Maths exam" (case-insensitive) | Pass |
| 20 | Delete missing task | `delete physics` | "I couldn't find a task" | Pass |
| 21 | Delete bad number | `delete 9` | "There is no task number 9" | Pass |
| 22 | Delete with no target | `delete` | Asks which task to delete | Pass |
| 23 | Delete by number | `delete 1` | "Deleted: Lab record" | Pass |
| 24 | Alert fires once | a task due 5 seconds ago, checked twice | 1 alert, then 0 | Pass |

In the GUI run, a task saved 25 seconds ahead produced the REMINDER bubble and the pop-up on time (the beep depends on the computer's sound settings), and `tasks.txt` recorded it as alerted.

# 7. Conclusion

Calendar ChatBot meets all its objectives: users can schedule, list and delete tasks by chatting, invalid input is handled gracefully, tasks persist in a file, and alerts appear on time. The project applied the main course topics in one working program: classes and objects, encapsulation, inheritance, the Collections Framework, enums, Swing event handling, multithreading, exception handling, file I/O and the `java.time` API.

The most important design decision was keeping the logic (`ChatBot`) separate from the interface (`ChatWindow`). It made the logic testable without a window, which is how all 24 tests were automated.

**Limitations and future work**

- Reminders fire only while the app is open; a missed reminder is shown at the next start-up.
- Alerts can be up to 10 seconds late, because the checker runs every 10 seconds.
- Possible extensions: priority levels, recurring reminders, a snooze option, and editing an existing task.